Project Phase II
Pipelined RISC-V Processor Design

Course Project Report

Course: Intro to Processor Architecture

Submitted by

Name	**Roll Number**
Sindhuja Babu	<2024102055>
Aryaman Saran	<2024102027>
Lakshya Jindal	<2023112017>

International Institute of Information Technology, Hyderabad

March 2026

1 Introduction

Modern processors must execute a large number of instructions efficiently while maintaining high throughput. One of the most widely used techniques to improve processor performance is **pipelining**. Pipelining divides instruction execution into multiple stages so that several instructions can be processed simultaneously at different stages of execution.
In this project, we design and implement a **five-stage pipelined RISC-V processor** in Verilog. The processor follows the classical pipeline organization consisting of the following stages:

- Instruction Fetch (IF)
- Instruction Decode (ID)
- Execute (EX)
- Memory Access (MEM)
- Write Back (WB)

Although pipelining improves instruction throughput, it introduces several challenges. In particular, **data hazards** and **control hazards** arise when instructions depend on results that are not yet available or when branch instructions alter the control flow of the program.
To ensure correct execution, the processor incorporates several hazard handling mechanisms, including:

- Data forwarding to resolve read-after-write dependencies
- Load-use hazard detection and pipeline stalling
- Dynamic branch prediction to reduce control hazard penalties
- Pipeline flushing to recover from branch mispredictions

The processor was implemented in Verilog and verified through simulation using **Icarus Verilog**. Waveform analysis using **GTKWave** was used to observe pipeline behavior and validate the correct functioning of hazard handling mechanisms.

1.1 Objectives

The primary objective of this project is to design and implement a functional pipelined RISC-V processor and evaluate the mechanisms required to handle hazards introduced by pipelining.
The specific goals of the project are as follows:

- Implement a five-stage pipelined RISC-V processor consisting of IF, ID, EX, MEM, and WB stages.
- Design and integrate pipeline registers to allow multiple instructions to be processed concurrently.
- Implement a forwarding mechanism to resolve data hazards without introducing unnecessary pipeline stalls.
- Develop a hazard detection unit to detect load-use dependencies and appropriately stall the pipeline.
- Implement a dynamic branch prediction mechanism to reduce control hazard penalties.
- Design pipeline flushing logic to recover from branch mispredictions.
- Verify the correct functionality of the processor through simulation and waveform analysis.

2 Processor Architecture

The processor follows a classical **five-stage pipelined architecture**, allowing multiple instructions to be processed simultaneously. Intermediate results are held in pipeline registers between stages.

2.1 Instruction Fetch (IF)

The PC is used to fetch instructions from instruction memory, incrementing by 4 each cycle. A **dynamic branch predictor** determines the next PC on branch instructions. The fetched instruction and PC are stored in the **IF/ID pipeline register**.

2.2 Instruction Decode (ID)

The instruction is decoded, control signals are generated, source operands are read from the register file, and immediate values are extracted. Hazard detection logic identifies load-use hazards at this stage. Results are passed to the **ID/EX pipeline register**.

2.3 Execute (EX)

The ALU performs arithmetic and logical operations, and branch conditions are evaluated. A **forwarding unit** supplies up-to-date operands directly from later stages, reducing data hazard stalls.

2.4 Memory Access (MEM)

Load and store instructions access data memory here. For all other instructions, the ALU result passes through unchanged.

2.5 Write Back (WB)

The final result — either the ALU output or data read from memory — is written back to the register file.

3 Pipeline Registers

In a pipelined processor, pipeline registers are used to store intermediate results between stages so that multiple instructions can be processed simultaneously. Each pipeline register captures the outputs of one stage and provides them as inputs to the next stage during the next clock cycle.
The processor implemented in this project uses four pipeline registers:

- IF/ID
- ID/EX
- EX/MEM
- MEM/WB

These registers also support additional control mechanisms such as **stalling** and **flushing**, which are necessary for correct hazard handling.

3.1 IF/ID Pipeline Register

The **IF/ID register** transfers the fetched instruction and program counter value from the Instruction Fetch stage to the Instruction Decode stage.

- **Inputs:** PC and fetched instruction
- **Outputs:** PC and instruction forwarded to the decode stage

This register also supports pipeline control through the `stall` and `flush` signals.

- When `reset` or `flush` is asserted, the outputs are cleared, effectively inserting a **bubble** into the pipeline.
- When `stall` is asserted, the register holds its current values and prevents new instructions from entering the pipeline.

3.2 ID/EX Pipeline Register

The **ID/EX register** transfers decoded instruction information from the Instruction Decode stage to the Execute stage. The following information is stored in this register:

- Program counter value
- Instruction
- Register operand values
- Immediate value
- Source and destination register indices
- Control signals such as MemRead, MemWrite, RegWrite, ALUSrc, and Branch
- Branch prediction information (`pred_taken` and prediction index)

This register also handles hazard-related control signals:

- When `reset` or `flush` is asserted, all outputs are cleared to insert a bubble into the pipeline.
- When `stall` is asserted, the control signals are cleared while the data signals are preserved. This effectively inserts a **pipeline bubble** while maintaining the correct data flow.

Additionally, branch prediction information is forwarded to the Execute stage through this register so that the processor can later determine whether the prediction was correct.

3.3 EX/MEM Pipeline Register

The **EX/MEM register** transfers results from the Execute stage to the Memory stage. The outputs of the ALU and associated control signals are stored in this register.
The stored values include:

- ALU computation result
- Data to be written to memory for store instructions
- Destination register index
- Memory control signals such as MemRead and MemWrite
- Write-back control signals such as MemtoReg and RegWrite

3.4 MEM/WB Pipeline Register

The **MEM/WB register** transfers the final results from the Memory stage to the Write Back stage.
This register stores:

- Data read from memory
- ALU result
- Destination register index
- Control signals required for write-back (MemtoReg and RegWrite)

During the Write Back stage, either the memory data or the ALU result is written back to the register file depending on the value of the `MemtoReg` signal.

4 Control and Datapath Flow

This section describes how instructions flow through the pipeline stages, with pipeline registers passing data and control signals from one stage to the next on each clock cycle. Control signals are generated in the Instruction Decode stage from the instruction opcode. These signals travel through the pipeline registers and govern the behavior of the Execute, Memory, and Write Back stages.
The Program Counter (PC) increments by 4 each cycle to fetch the next sequential instruction. On branch instructions, the dynamic branch predictor supplies the next PC. If the prediction is wrong, the PC is corrected and any incorrectly fetched instructions are flushed from the pipeline.
In the Write Back stage, either the ALU result or the memory read data is written to the destination register, selected by the `MemtoReg` signal.

5 Hazard Handling Mechanisms

The use of pipelining allows multiple instructions to execute concurrently, but it introduces hazards when instructions interact across pipeline stages. In this processor, mechanisms have been implemented to handle both **data hazards** and **control hazards**. These mechanisms include forwarding, hazard detection with pipeline stalling, dynamic branch prediction, and pipeline flushing.

5.1 Forwarding Unit

Data hazards occur when an instruction requires the result of a previous instruction that has not yet been written back to the register file. To minimize stalls caused by such dependencies, a forwarding mechanism is implemented in the Execute stage.

The forwarding logic compares the source registers of the instruction currently in the Execute stage with the destination registers of instructions in later pipeline stages. If a match is detected, the value is forwarded directly to the ALU inputs.
The forwarding decision is implemented using the following logic:

Listing 1: Forwarding Logic for Operand A

```
// Forward from EX/MEM stage
if (EX_MEM_RegWrite && (EX_MEM_rd != 0) &&
    (EX_MEM_rd == ID_EX_rs1))
    ForwardA = 2'b10;

// Forward from MEM/WB stage
else if (MEM_WB_RegWrite && (MEM_WB_rd != 0) &&
         (MEM_WB_rd == ID_EX_rs1))
    ForwardA = 2'b01;

else
    ForwardA = 2'b00;
```

A similar comparison is performed for the second ALU operand. This mechanism allows the processor to forward values from the EX/MEM and MEM/WB stages, avoiding unnecessary stalls caused by read-after-write dependencies.

5.2 Hazard Detection and Pipeline Stalling

Although forwarding resolves many hazards, it cannot resolve load-use dependencies. In such cases, the result of a load instruction becomes available only after the Memory stage. If the next instruction attempts to use this value, the pipeline must be stalled.
To detect this condition, a hazard detection unit monitors the pipeline registers and generates a stall signal when necessary. The implemented logic is shown below:

Listing 2: Load-Use Hazard Detection Logic

```
if (ID_EX_MemRead &&
    (ID_EX_rd != 5'd0) &&
    ((ID_EX_rd == IF_ID_rs1) ||
     (ID_EX_rd == IF_ID_rs2)))
    stall = 1'b1;
else
    stall = 1'b0;
```

When a stall is asserted, the Program Counter and IF/ID pipeline register are prevented from updating, effectively inserting a pipeline bubble and allowing the load instruction to complete before the dependent instruction proceeds.

5.3 Control Hazards and Static Branch Resolution

Branch instructions introduce control hazards because the next instruction address depends on the outcome of the branch condition. In this design, branch conditions are evaluated in the Execute stage using ALU comparisons.
If the branch condition is satisfied, the processor redirects execution to the computed branch target address. Without prediction, the processor effectively follows a **static not-taken strategy**, where instructions continue to be fetched sequentially until the branch outcome is known.

5.4 Dynamic Branch Prediction (Optimization)

To reduce the performance penalty associated with control hazards, the processor implements a dynamic branch predictor based on a Branch History Table (BHT). Each entry in the BHT contains a 2-bit saturating counter that records the recent behavior of a branch.
The prediction is generated during the Instruction Fetch stage using the following logic:

Listing 3: Dynamic Branch Prediction Logic

```
wire [1:0] counter_if = bht[index_if];

assign predicted_taken =
       valid_if &
       is_branch_if &
       counter_if[1];
```

If the prediction indicates that the branch is taken, the predicted target address is computed using the branch immediate and used as the next Program Counter value.
Once the branch outcome is resolved in the Execute stage, the BHT entry is updated using a 2-bit saturating counter mechanism to improve future predictions.

5.5 Pipeline Flush Mechanism

When the predicted branch outcome differs from the actual outcome, the instructions fetched based on the incorrect prediction must be removed from the pipeline.
This processor uses a simple flush unit to perform this operation. When a misprediction occurs, the following signals are generated:

Listing 4: Pipeline Flush Logic

```
assign flush_IF_ID = mispredict;
assign flush_ID_EX = mispredict;
```

These signals clear the corresponding pipeline registers, removing incorrectly fetched instructions from the pipeline. The Program Counter is then updated with the correct target address, allowing execution to resume along the correct control flow path.

6 Waveform Verification

To verify the correct operation of the pipelined processor and the implemented hazard handling mechanisms, simulations were performed using Icarus Verilog. The resulting execution traces were analyzed using GTKWave.
Dedicated test programs were created to trigger different pipeline scenarios, including data forwarding, load-use hazards, branch prediction, and branch misprediction recovery. For each test case, relevant internal signals were monitored to confirm that the processor behaves as expected.

6.1 Forwarding Verification

To verify the forwarding mechanism, a test program containing a data dependency between two consecutive instructions was executed.
Instruction Sequence

```
add x1, x2, x3
sub x4, x1, x5
```

The second instruction requires the value of register `x1` produced by the first instruction before it has been written back to the register file. Without forwarding, the pipeline would need to stall.
The waveform in Figure 1 shows the signals used to verify forwarding:

- `ForwardA`, `ForwardB`
- `EX_MEM_rd`, `MEM_WB_rd`
- `ID_EX_rs1`, `ID_EX_rs2`

The forwarding control signals use the following encoding:

- `00`: Operand from register file
- `01`: Forward from MEM/WB stage
- `10`: Forward from EX/MEM stage

In the waveform, `ForwardA` first becomes `01`, indicating that the operand is forwarded from the MEM/WB stage when the destination register matches `ID_EX_rs1`. In the following cycle, `ForwardA` becomes `10`, showing forwarding from the EX/MEM stage. The signal `ForwardB` also becomes `10`, confirming forwarding for the second ALU operand.
These observations confirm that the forwarding unit correctly resolves read-after-write (RAW) hazards without introducing pipeline stalls.

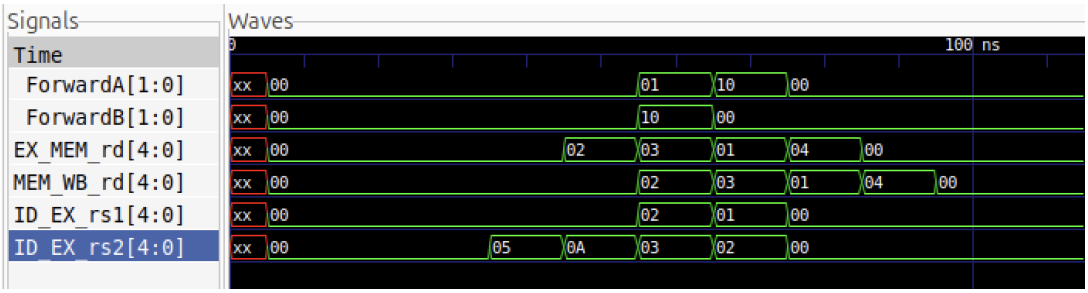

Figure 1: GTKWave waveform demonstrating operand forwarding from EX/MEM and MEM/WB stages.

6.2 Load-Use Hazard Stall Verification

To verify the hazard detection unit, a test case was constructed where an instruction immediately following a load instruction attempts to use the loaded value.

Instruction Sequence

```
ld  x1, 0(x5)
add x2, x1, x3
```

In this case, the `add` instruction requires the value of register `x1` that is produced by the preceding load instruction. Since the loaded data becomes available only after the Memory stage, forwarding alone cannot resolve this dependency and the pipeline must stall.

The waveform in Figure 2 shows the signals involved in hazard detection:

- `ID_EX_MemRead`
- `ID_EX_rd`
- `IF_ID_rs1`, `IF_ID_rs2`
- `stall`
- `pc`

When the load instruction reaches the Execute stage, the signal `ID_EX_MemRead` becomes high and the destination register `ID_EX_rd` is compared with the source registers of the instruction in the decode stage. In the waveform, `ID_EX_rd` matches `IF_ID_rs1`, indicating that the following instruction depends on the value being loaded.

As a result, the hazard detection unit asserts the `stall` signal. During this cycle, the Program Counter remains unchanged, which can be observed in the waveform. This confirms that the pipeline is temporarily frozen, inserting a bubble and allowing the load instruction to complete before the dependent instruction proceeds.

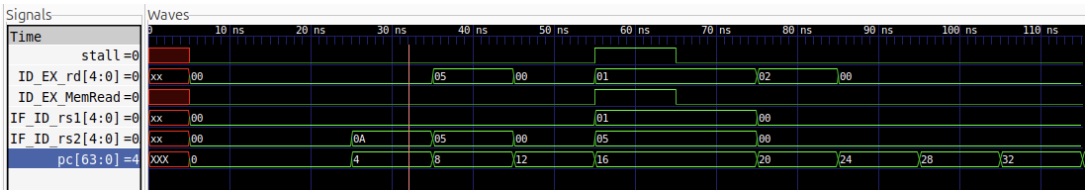

Figure 2: GTKWave trace showing a load-use hazard where the stall signal is asserted and the program counter remains unchanged.

6.3 Branch Prediction Verification

To evaluate the dynamic branch predictor, a test program containing a conditional branch instruction was executed.

Instruction Sequence

```
addi x1, x0, 1
addi x2, x0, 1
beq  x1, x2, 8
add  x3, x0, x0
```

The signals monitored in the waveform include:

- `pc`
- `predicted_taken`
- `predicted_pc_if`
- `pred_index_if`

The predictor uses a Branch History Table (BHT) indexed using bits of the program counter. Each entry stores a 2-bit saturating counter that determines whether a branch is predicted as taken or not taken.

Since all BHT entries are initialized to the state `01` (weakly not taken), the branch is initially predicted as not taken. This behavior can be observed in the waveform where the predicted next program counter (`predicted_pc_if`) always corresponds to the sequential address `pc + 4`.

The signal `pred_index_if` shows the index used to access the BHT, which increases sequentially as the program counter advances.

These observations confirm that the branch predictor correctly computes the BHT index and produces predictions based on the stored saturating counter values.

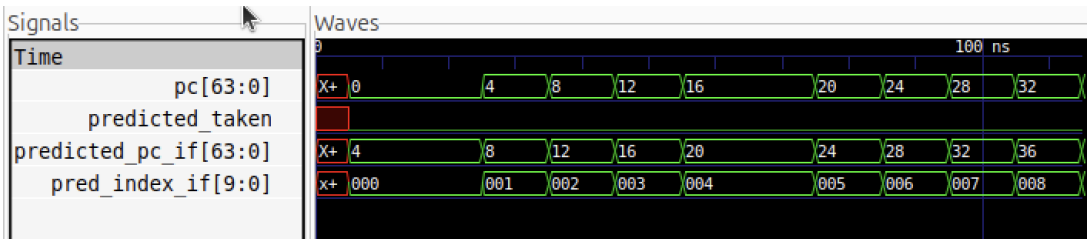

Figure 3: Waveform demonstrating branch prediction using the Branch History Table.

6.4 Branch Misprediction and Pipeline Flush Verification

To verify the control hazard handling mechanism, a test program containing a conditional branch instruction was executed. The program was designed such that the branch condition evaluates to true, while the branch predictor initially assumes a not-taken outcome.

Instruction Sequence

```
addi x1, x0, 1
addi x2, x0, 1
addi x3, x0, 5
beq  x1, x2, 12
add  x4, x0, x0
add  x5, x0, x0
add  x6, x3, x3
add  x7, x6, x1
```

Since the Branch History Table is initialized to the state `01` (weakly not taken), the branch predictor initially predicts that the branch will not be taken. However, because the condition `x1 == x2` evaluates to true, the branch is actually taken.

The waveform in Figure 4 shows the relevant signals involved in detecting and recovering from the misprediction:

- `predicted_taken`
- `branch_taken`
- `mispredict`
- `flush_IF_ID`
- `flush_ID_EX`
- `pc`

When the branch instruction reaches the Execute stage, the actual branch outcome becomes available. In the waveform, `predicted_taken` is low while `branch_taken` becomes

high, indicating that the prediction was incorrect. This mismatch causes the signal `mispredict` to be asserted.
As a result, the flush signals `flush_IF_ID` and `flush_ID_EX` become active, clearing the corresponding pipeline registers and removing the incorrectly fetched instructions. The program counter is then redirected to the correct branch target, after which normal pipeline execution resumes.

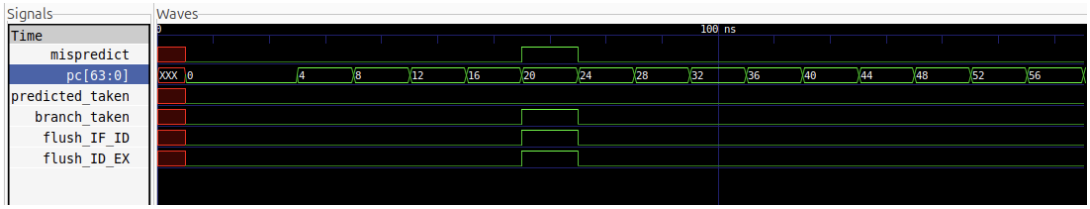

Figure 4: Waveform demonstrating branch misprediction detection and pipeline flush.

7 Performance Analysis

The performance of the pipelined processor was evaluated by measuring the total number of clock cycles required to execute a test program containing arithmetic, logical, memory, and branch instructions. The program consists of 18 instructions and completed execution in 21 clock cycles.
The resulting cycles-per-instruction (CPI) is therefore

$$CPI = \frac{\text{Total Cycles}}{\text{Number of Instructions}} = \frac{21}{18} \approx 1.17$$

In an ideal five-stage pipeline, the CPI approaches 1 once the pipeline is filled. The slightly higher CPI observed here is due to pipeline fill and drain overhead, load-use hazards, and branch misprediction penalties.
Most data hazards in the program are resolved using the forwarding mechanism implemented in the Execute stage, which allows intermediate results from later pipeline stages to be directly supplied to the ALU inputs of subsequent instructions. However, load-use hazards cannot be resolved through forwarding alone, and the hazard detection unit inserts a stall cycle when necessary to ensure correct execution.
Control hazards introduced by branch instructions are mitigated using a dynamic branch predictor. When a misprediction occurs, the pipeline flush mechanism removes incorrectly fetched instructions and redirects execution to the correct branch target.
The instruction throughput achieved by the processor is

$$Throughput = \frac{18}{21} \approx 0.86 \text{ instructions per cycle}$$

8 Conclusion

In this project, a five-stage pipelined RISC-V processor was implemented in Verilog using the classical IF, ID, EX, MEM, and WB pipeline stages. Pipeline registers were introduced to enable concurrent execution of multiple instructions.

To ensure correct execution in the presence of hazards, several mechanisms were incorporated. Data hazards were mitigated using operand forwarding, load-use hazards were handled through a hazard detection unit that inserts stalls when necessary, and control hazards were addressed using a dynamic branch predictor combined with a pipeline flush mechanism.

The design was verified through simulation and waveform analysis using GTKWave, confirming correct pipeline behavior and proper handling of hazards. Performance evaluation shows that the processor achieves a CPI close to the ideal value, demonstrating improved instruction throughput while maintaining correct program execution.